Department of Computer Science & Engineering
The University of Texas at Arlington

Architectural Design Specification
CSE 4317: Senior Design II
Fall 2025

IGVC
Mav Truck

Alexander Kowis
Thanh Tran
Brian Biju
Khanh Van Lam
Kopil Sharma
Rezwana Roza

Revision History

Revision	Date	Author(s)	Description
0.1	09.18.2025	KS, AK, RR, TT, KV	document creation
0.2	09.19.2025	KS	Completed intro, Subsystems
0.3	09.19.2025	AK	Completed Camera Subsection of Input Layer
0.4	09.19.2025	KS, AK, RR, TT, KV	Completed input output and processing layers

Contents

1 Introduction **5**

2 System Overview **6**
2.1 Input Layer Description 6
2.2 Output Layer Description 6
2.3 Processing Layer Description 7

3 Subsystem Definitions & Data Flow **8**

4 Input Layer **9**
4.1 Camera 9
4.2 LIDAR 10
4.3 Optical Encoders 11
4.4 Emergency Switches 12
4.5 GPS 13

5 Output Layer **14**
5.1 Motor Driver Control 14
5.2 Lights Control 15

6 Processing Layer **16**
6.1 JETSON 16
6.2 ROS 2 17
6.3 MICROCONTROLLER 17

List of Figures

1 IGVC simple architectural diagram 6
2 Data flow of our layers. 8
3 Input Layer subsystem 9
4 Camera Subsection on Data Flowchart 10
5 Output Layer subsystems 14
6 Pocessing Layer Subsystem 16
7 Microcontroller subsystem 18

List of Tables

2 Camera Interfaces 10
3 LIDAR Interfaces 11
4 Optical Encoder Interfaces 12
5 Emergency Stop Switches 13
6 GPS Interfaces 13
7 Subsystem interfaces 15
8 Lights Subsystem Interfaces 15
9 Jetson interfaces 17
10 Microcontroller Interfaces 18

1 Introduction

Our project is an autonomous ground vehicle designed for the Intelligent Ground Vehicle Competition (IGVC). The vehicle integrates multiple sensors and safety mechanisms to perceive its environment and make navigation decisions. Specifically, the inputs to the system include a camera, LIDAR, GPS, optical encoders, and emergency safety switches. These inputs provide information such as lane markings, obstacle positions, global location, wheel odometry, and emergency stop conditions. Together, these inputs ensure that the vehicle has sufficient environmental awareness and safety monitoring to operate under IGVC conditions.

The core processing system is built around the NVIDIA Jetson Orin Nano, running Ubuntu and ROS2. This processing layer fuses incoming sensor data in real time, performs perception tasks such as lane detection and obstacle avoidance, carries out localization using GPS and encoder feedback, and executes path planning. Once navigation decisions are made, the Jetson issues control commands to the actuation subsystems.

The outputs of the system consist of drive motors and lights. Motor commands are sent through the Generic high-power DC motor driver, which translates high-level velocity and direction inputs into differential drive signals for the left and right wheels. The lights subsystem provides visual signals to indicate system status, activity, and emergency conditions, ensuring both compliance with IGVC requirements and safety for operators and bystanders.

The overall design follows a layered architecture to ensure modularity, clarity, and reliability. Layer X represents the input layer, which collects and organizes raw environmental and safety data. Layer Z is the processing layer, where data fusion, perception, localization, planning, and decision-making occur. Finally, Layer Y is the output layer, where commands from the processing layer are converted into physical actions such as movement and signaling. This separation of concerns allows for clear interfaces between subsystems, easier debugging, and safer operation.

By combining these sensors, processing units, and actuators within a structured layered design, the system provides a complete platform for autonomous navigation in outdoor environments. The architecture emphasizes modular development, real-time responsiveness, and safety, which are essential for successful participation in the Intelligent Ground Vehicle Competition. [1] [2] [3] [4]

2 System Overview

The system is organized into three main layers to separate sensing, processing, and actuation. Layer X (Input Layer) collects data from sensors including the camera, LIDAR, GPS, encoders and emergency switches. Layer Z (Processing Layer), built around the Jetson Orin Nano running Ubuntu and ROS2, fuses sensor data to perform perception, localization, and path planning. Finally, Layer Y (Output Layer) converts these decisions into physical actions through the motors for movement and lights for signaling. This layered design ensures clear responsibilities, modular development, and reliable real-time performance

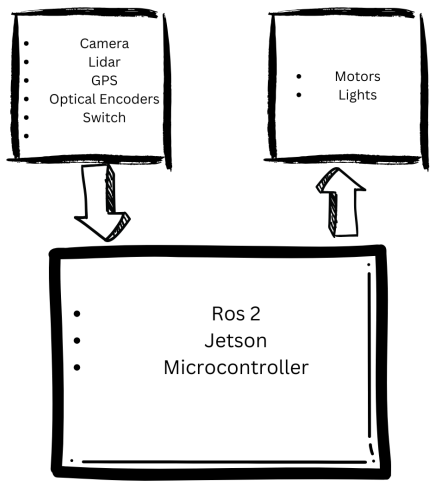

Figure 1: IGVC simple architectural diagram

2.1 Input Layer Description

Layer X is the input layer and is responsible for collecting all raw environmental and safety data. Its subsystems include the camera, LIDAR, GPS, encoders and emergency switches, each of which provides a unique stream of information. The main functions of this layer are to capture images, generate point clouds, provide positional fixes, and send stop signals, which are then passed to the processing layer in a consistent and usable format. Critical interfaces include USB, serial, microcontroller and GPIO connections between the sensors and the microcontroller and the orin nano. The services provided by this layer include synchronized and time-stamped data streams that support perception, localization, and navigation. Conventions include using clear naming for sensor data (such as camera, lidar, gps) and assigning each sensor a defined reference frame to ensure modularity and consistency across subsystems.

2.2 Output Layer Description

Layer Y is the output layer, which converts the systemâs decisions into physical actions. This layer consists mainly of the motors and lights. The motor subsystem translates velocity and direction commands into differential drive signals through the Generic high-power DC motor driver, while the lights subsystem provides system visibility and safety signaling. The critical interfaces for this layer include UART for motor commands and GPIO or microcontroller links for controlling the lights. Services provided by this layer include reliable propulsion, visual indicators of system state, and safety enforcement through emergency stop overrides. Conventions include using consistent control signals and topic names for commands (such as motor velocity and light state) to ensure clear communication between layers.

2.3 Processing Layer Description

Layer Z is the processing layer and serves as the core of the decision-making system. Built around the Jetson Orin Nano running Ubuntu and ROS2, this layer fuses inputs from the camera, LIDAR, GPS, encoders, microcontroller and switches to create a real-time understanding of the environment. Its functions include perception, localization, path planning, and command generation for the output layer. Critical interfaces include ROS2 communication channels that connect sensor inputs from Layer X and actuator commands to Layer Y, along with safety channels for emergency stop signals. Services provided by this layer include accurate and timely decision-making and safe navigation.

3 Subsystem Definitions & Data Flow

This section breaks down your layer abstraction to another level of detail. Here you grapically represent the logical subsytems that compose each layer and show the interactions/interfaces between those subsystems. A subsystem can be thought of as a programming unit that implements one of the major functions of the layer. It, therefore, has data elements that serve as source/sinks for other subsystems. The logical data elements that flow between subsystems need to be explicitly defined at this point, beginning with a data flow-like diagram based on the block diagram.

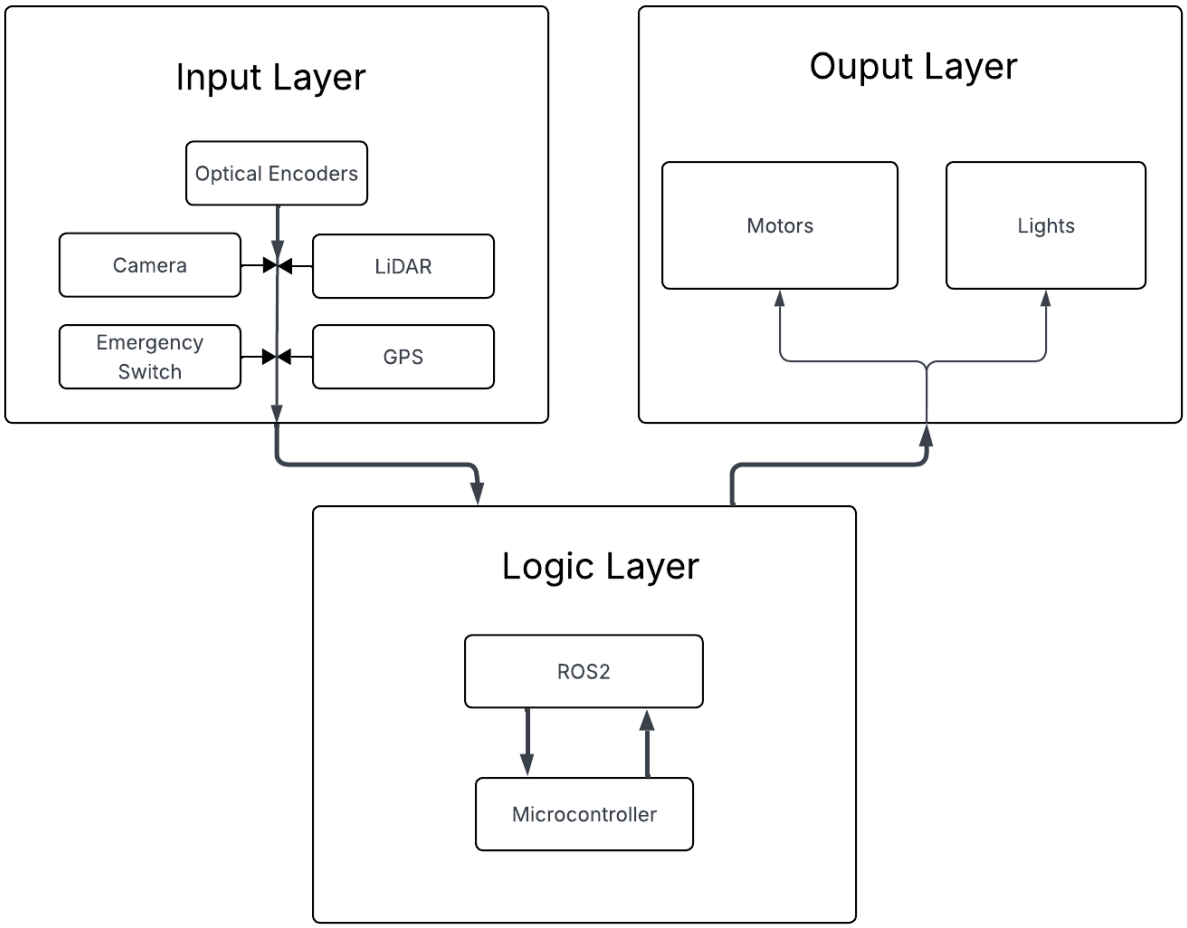

Figure 2: Data flow of our layers.

4 Input Layer

SAThis layer describes the inputs of our system as a whole. These inputs provide data through our logic layer, for decision making to move our vehicle. Our inputs include the following: Camera, LiDAR, Optical Encoders, Emergency Switch, and GPS.

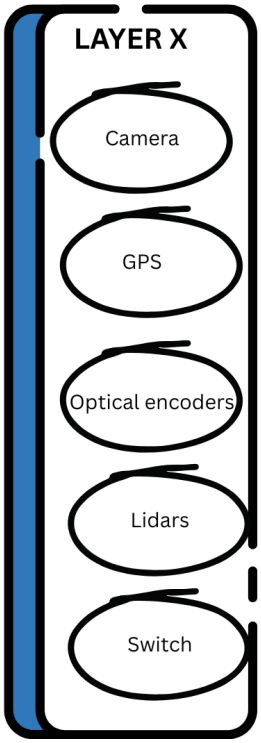

Figure 3: Input Layer subsystem

4.1 Camera

The camera we are testing right now is the Logitech C920 webcam. There are 2 main objectives of the camera: lane detection and image detection. The data passed form these algorithms will assist our vehicle in decision making while completing the course. Our vehicle will need to stay within the lanes, and make moves depending on the images within the lanes. The camera will connected to and communicate with the Jetson Nano via USB.

4.1.1 Assumptions

It is assumed that the competition will be held outside during the daytime. This will affect the sensitivity of the camera and thresholds within the image processing. It is also assumed that the camera will be mounted towards the front of the vehicle with a clear view of the objectives in front of it.

4.1.2 Responsibilities

The camera will need to work hand-in-hand with the LiDAR when making decisions on how the vehicle will move. The LiDAR is used for object-detection, so any "objects" the LiDAR cannot detect, the camera will pick-up. This includes flat objects, like colored dots, and arrows/lines made with paint or tape. The camera will process the images and location/position algorithms will determine which way the vehicle will move. This data will be passed to the Jetson Nano and output directions for the motors.

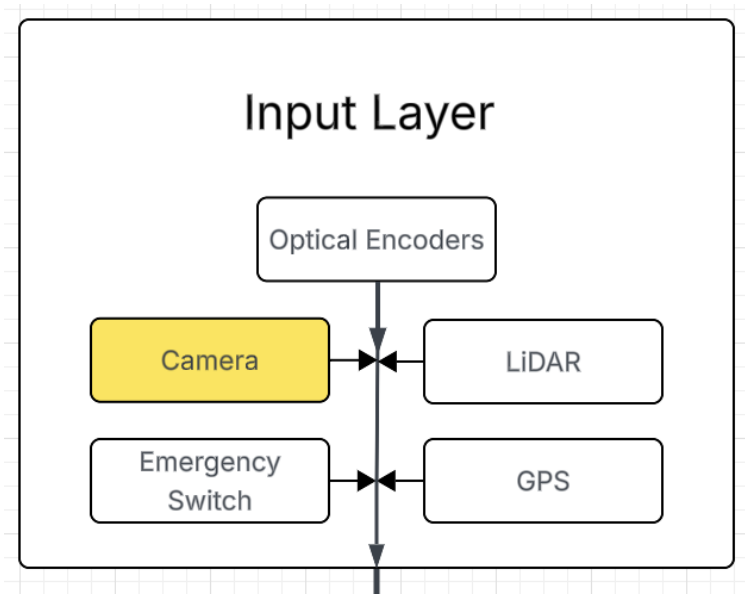

Figure 4: Camera Subsection on Data Flowchart

4.1.3 Image Processing Algorithms

The camera's job is to be part of the eyes of the vehicle. The camera will be looking for lanes, colored dots, turning arrows, and stop signs. This will all be done through location/position algorithms. The lane detection will be done through a different algorithm than the other 3 objects.

The lane detection will be done through image filtering, blurring, and edge detections. Once the lines are located, their slope will be calculated and the midpoint between them will be found. Using this midpoint, the algorithm will tell the Jetson Nano how far left or right the vehicle will need to turn to stay in between the lanes.

For image detection of the other 3 objects, the camera will locate one of the objects once it comes into frame. The distance of the object will be calculated and once it is within range, it will provide the Jetson Nano with directions on how the motors should turn.

4.1.4 Interfaces

Table 2: Camera Interfaces

ID	Description	Inputs	Outputs
#L1	USB Data Link	Images from video feed	Motor direction/speed / ROS2 topic

4.2 LIDAR

The LIDAR subsystem provides the information on if there are any obstacles around by scanning the environment with laser pulses and returning distance measurements. For this project, we are using the RPLIDAR S3 M1, which generates a 360-degree point cloud around the vehicle. The data collected is essential for detecting obstacles, identifying free space, and aiding in path planning. The LIDAR

communicates with the Jetson Orin Nano through a USB interface, and the information is published into ROS2 for real-time processing alongside other sensor inputs.

4.2.1 Assumptions

It is assumed that the LIDAR has a clear field of view around the vehicle and is mounted securely to reduce vibration. The LIDAR provides range data at sufficient frequency (5â15 Hz) and resolution for obstacle avoidance. It is also assumed that the ROS2 driver package is compatible with the hardware and properly calibrated to align with the vehicleâs coordinate frame.

4.2.2 Responsibilities

The responsibilities of this subsystem are to continuously collect 360-degree range measurements, detect the angle the obstacle is on and send it for furthur processing to the jetson. The data supports obstacle detection and navigation algorithms.

4.2.3 Interfaces

Table 3: LIDAR Interfaces

ID	Description	Inputs	Outputs
#L1	USB Data Link	Laser reflections from environment	distance and angle to jetson / ROS2 topic
#L2	Power Supply	5V DC input	Operational power to LIDAR unit

4.3 Optical Encoders

The Optical Encoder subsystem provides wheel rotation feedback to estimate odometry. Encoders are mounted on the drive wheels and measure angular displacement, which is converted into distance traveled. This information complements GPS and IMU data, especially in areas with poor GPS coverage. Due to encoders we boycott the IMU. The encoder signals are processed by a microcontroller and then sent to the Jetson Orin Nano for integration into the ROS2 navigation stack.

4.3.1 Assumptions

It is assumed that the encoders are properly calibrated to the wheel circumference and can measure rotation with adequate resolution for smooth odometry. It is also assumed that electrical noise is minimized through proper wiring and shielding, ensuring clean signal transmission to the microcontroller.

4.3.2 Responsibilities

The responsibilities of this subsystem are to provide continuous wheel rotation counts, calculate distance traveled, and publish odometry data. This data supports localization, path tracking, and closed-loop control of the vehicle. Encoders also provide redundancy in case of degraded GPS signals.

4.3.3 Interfaces

Table 4: Optical Encoder Interfaces

ID	Description	Inputs	Outputs
#E1	Encoder Signal Lines	Wheel rotation pulses (A/B channels)	Signals to microcontroller
#E2	Data Link	Processed counts from microcontroller	Odometry data to Jetson / ROS2 topic

4.4 Emergency Switches

The emergency stop system consists of two complementary switches: a mechanical kill switch and a wireless remote switch. The mechanical kill switch is a large, red, mushroom-style button mounted in an accessible location on the vehicle, designed to immediately cut motor power or disable the driver enable line through a safety relay when pressed. It latches in the stop position until manually reset by twisting or pulling it out, ensuring the vehicle cannot restart without operator intervention. In parallel, a wireless remote switch allows the operator to stop the vehicle from a distance by transmitting a coded signal to a receiver onboard. The receiver triggers the same emergency stop circuit as the mechanical switch, ensuring motors are disabled instantly. To maintain safety, the wireless link uses basic validation to prevent false triggers, and loss of communication defaults to a safe state where the vehicle remains disabled. Together, these switches provide redundant and reliable emergency stop functionality both locally and remotely.

4.4.1 Kill Switch

The vehicle uses a large, red, mushroom-style emergency stop switch mounted in an accessible location on the chassis. When pressed, the switch immediately interrupts the motor enable signal or cuts power through a safety relay. The action is latched until the operator manually resets the switch by twisting or pulling it back out. This ensures the vehicle remains disabled until it is safe to re-enable. The mechanical switch provides a direct, reliable, and highly visible emergency stop mechanism for on-site operators.

4.4.2 Wireless Remote Switch

A handheld wireless remote provides the operator the ability to stop the vehicle at a distance. The remote transmits a coded stop signal to a receiver mounted on the vehicle. Once received, the signal activates the same emergency stop circuit as the mechanical switch, disabling motor power or the driver enable line. The wireless link includes simple validation (e.g., paired channel, checksum) to prevent false triggers, and loss of communication defaults to a safe state where the motors remain disabled. This remote option ensures safety even when the operator cannot physically reach the onboard kill switch.

Table 5: Emergency Stop Switches

ID	Description	Inputs	Outputs
#J1	Mechanical Kill Switch (on-vehicle)	Operator press	Stop signal to disable motors
#J2	Wireless Remote Receiver (on-vehicle unit)	RF packet from handheld transmitter	Stop signal to disable motors

4.5 GPS

The GPS subsystem provides the vehicle with its absolute position, which is essential for localization and navigation during IGVC tasks. A compact GPS receiver is mounted on the chassis with a clear view of the sky. The receiver connects directly to the Tiva-C microcontroller, which continuously reads the GPS data and relays it to the Jetson computer. This setup ensures that raw GPS messages are validated and converted into a reliable data stream before reaching higher-level autonomy software.The subsystem is designed to supply latitude, longitude, altitude, and speed information at several updates per second.

4.5.1 Assumptions

The GPS receiver is connected to the Tiva-C microcontroller through a UART interface and provides position updates at a rate of 5â10 Hz. It is assumed that the antenna is mounted with a clear view of the sky so that the module can achieve stable satellite lock with typical accuracy of 2â3 meters. The Tiva-C is assumed to correctly power, read, and validate the GPS data before sending it to the Jetson computer over a serial link. It is also assumed that the Jetson fuses this GPS data with IMU readings for smoother localization.

4.5.2 Responsibilities

The GPS subsystem is responsible for supplying the robot with continuous position information in the form of latitude, longitude, altitude, and basic velocity. The Tiva-C collects the GPS data, validates it, and formats it into a reliable packet that the Jetson can interpret. On the Jetson, the data is used for localization and fused with IMU measurements to provide filtered odometry to the planning and control subsystems. The subsystem must also indicate when a GPS fix is weak or unavailable so that the system can safely rely on alternative sensors.

Table 6: GPS Interfaces

ID	Description	Inputs	Outputs
#G1	GPS receiver (Tiva-C)	Raw GPS data	Parsed GPS record
#G2	Tiva-C (Jetson serial link)	Parsed GPS record	Position packets to Jetson
#G3	Jetson fusion	GPS inputs	Filtered odometry

5 Output Layer

The Y layer is the output layer, responsible for carrying out the physical actions of the vehicle. It includes the motor driver, which translates velocity and direction commands into differential drive signals for the left and right motors, and the lights subsystem, which provides visual indicators for system status and safety. Together, these subsystems ensure reliable movement, signaling, and compliance with IGVC safety requirements

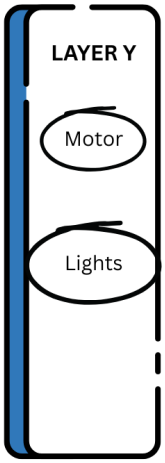

Figure 5: Output Layer subsystems

5.1 Motor Driver Control

This subsystem manages communication between the Jetson Orin Nano and the Sabertooth 2x32 motor driver. It translates velocity and direction commands from higher layers into motor actuation signals, controlling the left and right drive motors.

5.1.1 Assumptions

The Jetson Orin Nano communicates with the Sabertooth 2 x 32 motor driver through a UART serial interface using compatible commands. The motors are correctly wired and rated for the driver outputs, and higher layers (ROS2 or Jetson control) can issue emergency stop commands that immediately disable motor outputs.

5.1.2 Responsibilities

This subsystem converts velocity and direction commands from the Jetson into motor driver commands that control the speed and direction of the left and right motors for differential drive. It enables or disables motor outputs as required and handles emergency stop commands to immediately halt the motors.

5.1.3 Subsystem Interfaces

Each of the inputs and outputs for the subsystem are defined here. Create a table with an entry for each labelled interface that connects to this subsystem. For each entry, describe any incoming and outgoing data elements will pass through this interface.

Table 7: Subsystem interfaces

ID	Description	Inputs	Outputs
#M1	Motor control bus	Velocity direction	Left/Right motor actuation signals
#M2	Safety interrupt	Emergency stop signal	Disable outputs

5.2 Lights Control

The Lights subsystem provides visual indicators for vehicle status and safety. Lights are used to signal when the system is active, in motion, or in an emergency state. This subsystem ensures that the vehicle remains visible to operators, judges, and bystanders during operation and provides compliance with IGVC safety requirements.

5.2.1 Assumptions

It is assumed that the lights are powered by a stable DC source and is connected to the microcontroller for control. MOSFETS are controlling the lights through GPIO pins of the microcontroller. The microcontroller also communicates with the Jetson to get the cases for lights. The ROS2 control layer is expected to publish light commands, and safety overrides such as the emergency stop will take priority over normal signaling.

5.2.2 Responsibilities

The responsibilities of the lights subsystem include activating green, red or yellow light or special lights during emergency stops, and providing continuous illumination when required. It ensures that safety-related signals are always prioritized and that the vehicle is visible at all times during operation. This subsystem supports debugging, safety monitoring, and competition rule compliance.

5.2.3 Subsystem Interfaces

Table 8: Lights Subsystem Interfaces

ID	Description	Inputs	Outputs
#L1	Light control bus	Commands from Jetson/ROS2 or microcontroller	Light ON/OFF through mosfets
#L2	Safety override	Emergency stop signal	Force lights into emergency state

6 Processing Layer

The Z layer is the processing layer and serves as the decision-making core of the vehicle. It integrates the Jetson Orin Nano, ROS2 middleware, and the Tiva-C microcontroller. The Jetson fuses data from sensors to perform perception, localization, and path planning, while ROS2 manages communication between subsystems. The microcontroller handles low-level safety tasks such as emergency switches, lights, and GPS interfacing. This layer ensures accurate decisions, safe navigation, and coordination between input and output layers

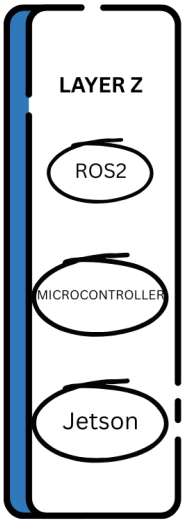

Figure 6: Pocessing Layer Subsystem

6.1 JETSON

The Jetson interfaces with sensors (camera, LIDAR, GPS), coordinates with ROS2 middleware, and communicates with the motor and lighting control subsystems through the microcontroller. It also manages safety functions such as issuing emergency stop signals.

6.1.1 Assumptions

The Jetson Orin Nano runs Linux with GPU acceleration for AI and vision tasks [3], using ROS2 for communication between subsystems. Sensors such as the camera, LIDAR, GPS, and switches provide compatible data, while a reliable UART, Ethernet, or USB link connects the Jetson to the microcontroller and motor driver. It is assumed to have sufficient processing power for real-time detection, avoidance, and navigation.

6.1.2 Responsibilities

The Jetson subsystem handles perception by processing sensor data into detections and localization, then uses this information for planning paths and generating velocity and direction commands for the motor driver. It coordinates inputs and outputs through ROS2, manages safety by issuing emergency

stop signals when hazards are detected, and records system data for debugging, evaluation, and validation.

6.1.3 Subsystem Interfaces

Each of the inputs and outputs for the subsystem are defined here. Create a table with an entry for each labelled interface that connects to this subsystem. For each entry, describe any incoming and outgoing data elements will pass through this interface.

Table 9: Jetson interfaces

ID	Description	Inputs	Outputs
#J1	Sensor Data Interface	Camera, LIDAR, GPS, switches	Perception
#J2	Control Command Bus	Navigation	Velocity, direction
#J3	Safety Channel	detection	Emergency stop

6.2 ROS 2

ROS 2 (Robot Operating System 2) provides the middleware framework that connects the vehicle's subsystems in a modular and scalable way. It uses a publish subscribe model built on DDS (Data Distribution Service) to enable efficient and real-time communication between nodes. For example, the LiDAR and camera nodes publish sensor data to topics, while the path-planning node subscribes to those topics to compute safe trajectories. In turn, the motor control node subscribes to velocity commands and converts them into motor driver signals. This architecture allows multiple processes to run concurrently and remain loosely coupled, improving both reliability and scalability.

6.3 MICROCONTROLLER

The microcontroller subsystem is implemented using a Tiva-C (TM4C123) board. It acts as the low-level controller for multiple hardware devices, including the GPS receiver, emergency switches, and indicator lights. The Tiva-C provides reliable, real-time handling of serial data and GPIO signals, ensuring that safety-critical functions (kill switch and lights) are always available, while also serving as the bridge between the GPS module and the Jetson computer. This subsystem, therefore, provides the foundation for safe operation and accurate data flow between the hardware layer and the autonomy software running on the Jetson.

6.3.1 Assumptions

It is assumed that the Tiva C microcontroller runs a lightweight embedded firmware that continuously polls or interrupts on its UART and GPIO lines. The GPS module communicates over UART at 3.3 V logic, while the lights and kill switch are handled through GPIO inputs and outputs. The Jetson Nano communicates with the Tiva C over USB serial, where the Tiva C presents itself as a virtual COM port device. Timing requirements are modest: GPS updates at 5-10 Hz, while light/switch signals are near instantaneous. The microcontroller is assumed to remain powered whenever the vehicle is active, with watchdog and reset mechanisms in place to prevent lockup.

6.3.2 Responsibilities

The microcontroller subsystem is responsible for three key functions. First, it manages the GPS receiver, reading position data, validating it, and transmitting formatted packets to the Jetson for use in localization. Second, it controls the emergency switches, reading the state of the physical mushroom

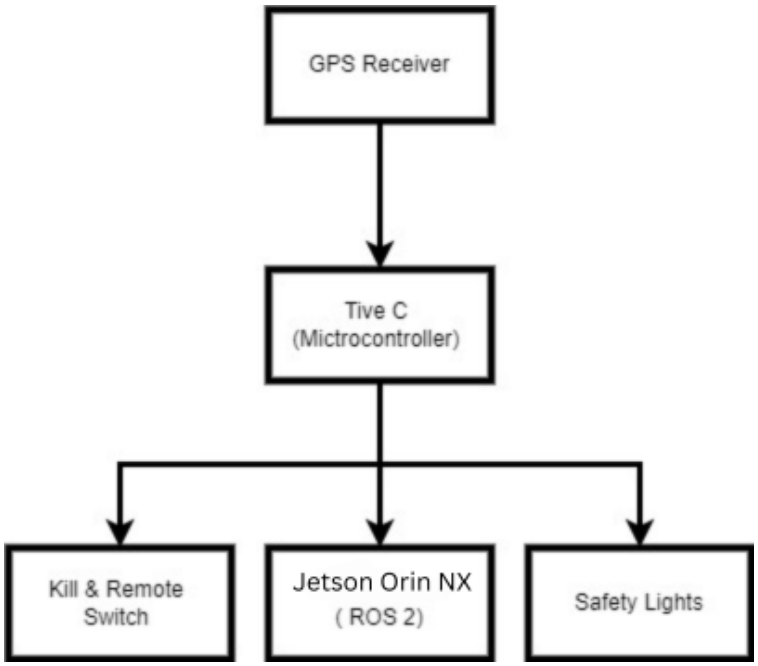

Figure 7: Microcontroller subsystem

kill switch and the wireless remote switch, and asserting a stop condition to the motor driver when triggered. Third, it manages the vehicle lights, turning them on or off according to safety status and operator commands. By consolidating these functions, the microcontroller subsystem ensures real-time safety handling while offloading repetitive tasks from the Jetson Nano.

6.3.3 Subsystem Interfaces

The microcontroller connects GPS, emergency switches, and lights to the Jetson. It reads GPS data over UART, monitors switch inputs through GPIO, and drives lights via output pins. All processed information is sent to the Jetson through a USB serial link, where it is used by the autonomy software.

Table 10: Microcontroller Interfaces

ID	Description	Inputs	Outputs
#MC1	GPS receiver	GPS position data	Formatted GPS packets to Jetson
#MC2	Emergency switch inputs	wireless stop	Kill signal to motor driver
#MC3	Indicator lights	Safety state	LEDs/Lights
#MC4	Tiva-C	Parsed GPS	USB Serial

References

[1] S. McDermott, "Motor maniacs: Autonomous ground vehicle senior design project," https://cse.uta.edu/senior-design/projects/motor-maniacs/, 2025, university of Texas at Arlington, posted May 5, 2025. Accessed: 2025-09-19.

[2] Intelligent Ground Vehicle Competition, "Igvc official website," http://www.igvc.org/, 2024, accessed: 2025-09-19.

[3] IGVC-ComputerVision, "igvc," 2024, gitHub repository.

[4] Debinb, "Igvc-robot-2024: Ads documentation (ads_igvc.pdf),"